

A Review of Line-Breaking Algorithms in Automatic Typesetting

FlashTeX Documentation Team

January 1, 1970

Abstract

This review surveys work on paragraph and page line-breaking algorithms as applied to automatic typesetting systems. We summarize the classical dynamic-programming approach to paragraph breaking (Knuth and Plass, 1981), subsequent refinements aimed at improving hyphenation quality (Liang, 1983), and more recent work on unified page and line breaking (Plass, 1981; Hobby, 1986; Frank, 1990). We conclude with a discussion of open problems in the area (Barron, 1995; Unicode Consortium, 2000).

1 Introduction

Automatic line breaking is one of the oldest and best-studied problems in digital typesetting. The dominant approach in wide use today traces back to Knuth and Plass (1981), who formulated paragraph breaking as a global optimization problem rather than a greedy, line-by-line process. Earlier systems had typically broken lines greedily, accepting whatever badness resulted from a locally optimal choice (Plass, 1981); the key insight of Knuth and Plass (1981) was that a small amount of lookahead, implemented via dynamic programming over candidate break points, could substantially reduce the overall unevenness of a paragraph's line lengths.

Hyphenation quality is closely tied to line-breaking quality: a poor hyphenation dictionary forces the algorithm to accept worse breaks purely for lack of a legal hyphenation point (Liang, 1983). The pattern-based approach introduced by Liang (1983) remains, in modified form, the basis of hyphenation in most modern systems, including several that are otherwise unrelated to the original T_EX implementation.

2 The Knuth–Plass Algorithm

Knuth and Plass (1981) model each potential break point with an associated *badness*, a nonnegative real number derived from how much the glue on a line must stretch or shrink to fill the line's measure. Formally, for a line whose natural width differs from the target width by an amount δ relative to the available stretch or shrink y , badness is approximated by

$$\beta(\delta, y) = 100 \left(\frac{\delta}{y} \right)^3,$$

clamped to an implementation-defined maximum representing an effectively infeasible line. The total cost of a paragraph is then the sum, over all chosen breakpoints, of a monotone function of badness plus a penalty term that discourages consecutive hyphenated lines and rewards visually similar adjacent line lengths (see Knuth and Plass, 1981, p. 14).

This total-cost formulation is what distinguishes the algorithm from the greedy approaches surveyed by (Plass, 1981): because the optimization considers the whole paragraph at once, a slightly worse choice on one line can be accepted if it yields much better choices on the following lines, a property that greedy algorithms cannot exploit by construction.

“The problem of breaking a paragraph into lines of approximately equal length, in such a way that the badness of the worst line is minimized, is naturally formulated as a shortest-path problem over a graph of candidate breakpoints.”

Later authors extended this framework beyond single paragraphs. Vertical, whole-page breaking was addressed by Frank (1990), who adapted the same dynamic-programming machinery to choose page breaks jointly with float placement, and by Hobby (1986), who considered the closely related problem of breaking curves into smooth piecewise segments using a similar optimality criterion (Hobby, 1986; Frank, 1990).

3 Extensions and Open Problems

Several extensions to the basic algorithm have been proposed over the years. Barron (1995) surveys adaptations for bidirectional and right-to-left scripts, where the notion of a “line” interacts with script directionality in ways the original formulation does not address. More recently, Unicode Consortium (2000) discusses how Unicode line-breaking classes interact with legacy hyphenation-pattern formats, an issue that remains only partially resolved in production typesetting systems (Unicode Consortium, 2000; Barron, 1995).¹

Open problems identified across this body of work (Knuth and Plass, 1981; Liang, 1983; Plass, 1981; Hobby, 1986; Frank, 1990; Barron, 1995; Unicode Consortium, 2000) include: extending the cost model to account for justified ragged-right text uniformly, integrating hyphenation quality directly into the per-breakpoint cost rather than treating it as a fixed penalty, and unifying page-breaking and float-placement decisions with paragraph breaking in a single global optimization rather than the current two-stage pipeline used by most implementations.

4 Conclusion

The dynamic-programming formulation of Knuth and Plass (1981) remains, four decades later, the reference point against which newer line-breaking algorithms are measured. While later work has extended the model to pages (Frank, 1990), curves (Hobby, 1986), and non-Latin scripts (Barron, 1995; Unicode Consortium, 2000), no single successor has displaced it as the default choice for high-quality paragraph typesetting.

References

- D. Barron. Line breaking for bidirectional and right-to-left scripts. *Journal of Digital Typography*, 12(3):145–162, 1995.
- A. Frank. Joint page breaking and float placement. *Proceedings of the Symposium on Document Processing*, pages 201–214, 1990.

¹A regularly updated summary of open issues is maintained online, though such resources tend to drift out of date faster than the peer-reviewed literature they summarize.

- J. D. Hobby. Smooth, easy to compute interpolating splines. *Discrete and Computational Geometry*, 1(1):123–140, 1986.
- D. E. Knuth and M. F. Plass. Breaking paragraphs into lines. *Software: Practice and Experience*, 11(11):1119–1184, 1981.
- F. M. Liang. *Word Hy-phen-a-tion by Com-put-er*. PhD thesis, Stanford University, 1983.
- M. F. Plass. *Optimal Pagination Techniques for Automatic Typesetting Systems*. PhD thesis, Stanford University, 1981.
- Unicode Consortium. Unicode line breaking properties and legacy hyphenation formats. *Technical Report*, Unicode Consortium, 2000.